

From Text to Meaning

Chapter 2 · Tokenization and an introduction to word embeddings

Dr. A. Belcaid

Table of contents

1 What is tokenization? 2
1.1 One string, several valid segmentations . . . 3
1.2 Token, type, and vocabulary . . . 3

2 Why not tokenize into characters? 3

3 Why not tokenize into words? 4
3.1 The granularity dilemma . . . 4

4 The modern compromise: subwords 5
4.1 The intuition behind byte-pair encoding . . . 5
4.2 Names you will encounter . . . 6

5 Tokenization in practice 6
5.1 A deliberately small regular-expression tokenizer . . . 6
5.2 A linguistic tokenizer with spaCy . . . 7
5.3 A pretrained model tokenizer . . . 7

6 Token IDs are not meanings 8

7 Embeddings: meaning as geometry 8
7.1 Measuring similarity . . . 10
7.2 The classical analogy . . . 10

8 Exploring pretrained embeddings with Gensim 10
8.1 Plot a small part of the embedding space . . . 11

9 What comes next? 12

10 Further reading and tools 12
10.1 Good reads . . . 12

10.2 Libraries used in this chapter 13

In the previous chapter, we treated “text” as if it could be handed directly to a model. It cannot. A model operates on numbers, so we need to make two decisions:

1. **Tokenization:** which pieces of text will be the model’s units?
2. **Representation:** which numbers will stand for those units?

This chapter develops the main ideas and leaves most implementation details to the libraries that solve them well.

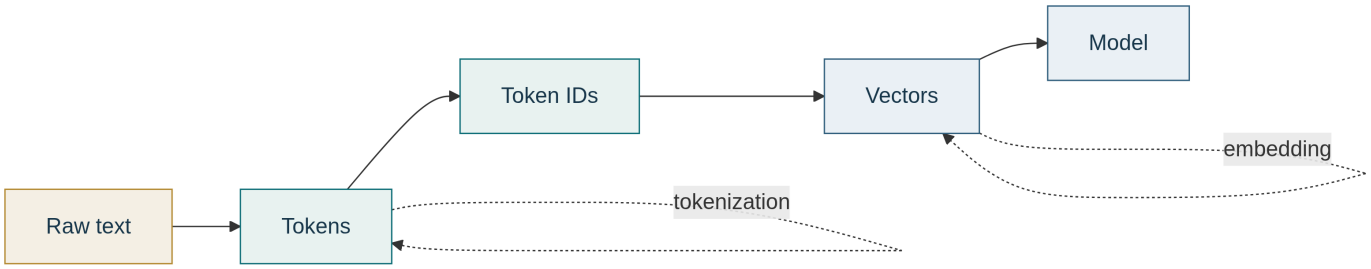


Figure 1: Our journey: choose units, assign identifiers, then represent those units in a space where useful relationships can emerge.

Learning objectives

After this chapter, you should be able to:

1. explain what tokenization does and why it is a modeling decision;
2. describe the limitations of character- and word-level tokenization;
3. explain why subwords are the dominant modern compromise;
4. inspect tokenization with regular expressions, spaCy, and a pretrained model tokenizer;
5. distinguish token identifiers, one-hot vectors, and dense embeddings; and
6. use pretrained embeddings to explore similarity and analogy.

1 What is tokenization?

Tokenization segments text into the units a model will process. Each observed unit is a **token**. The set of distinct units known to the model is its **vocabulary**.

```
Text       "NLP is fascinating!"
Tokens     ["NLP", "is", "fascinating", "!"]
Token IDs  [314, 27, 8921, 4]
```

The identifiers are vocabulary addresses. The number 8921 does not mean that *fascinating* is greater than *NLP*; it only points to a row in a table.

1.1 One string, several valid segmentations

What is the correct tokenization of `don't`?

```
["don't"]
["do", "n't"]
["don", "'", "t"]
["d", "o", "n", "'", "t"]
```

All four are mechanically possible. A linguistically informed system might preserve the contraction or separate its grammatical components. A subword model might choose reusable character sequences. A character model uses individual symbols.

> **!** Tokenization is not neutral cleanup. It determines the units the model can see, the vocabulary it must store, and the length of every input.

1.2 Token, type, and vocabulary

In the sentence:

> A model is a model.

there are six word/punctuation **tokens**, but fewer distinct **types** because `a` and `model` occur twice. A vocabulary contains types; an input contains token instances.

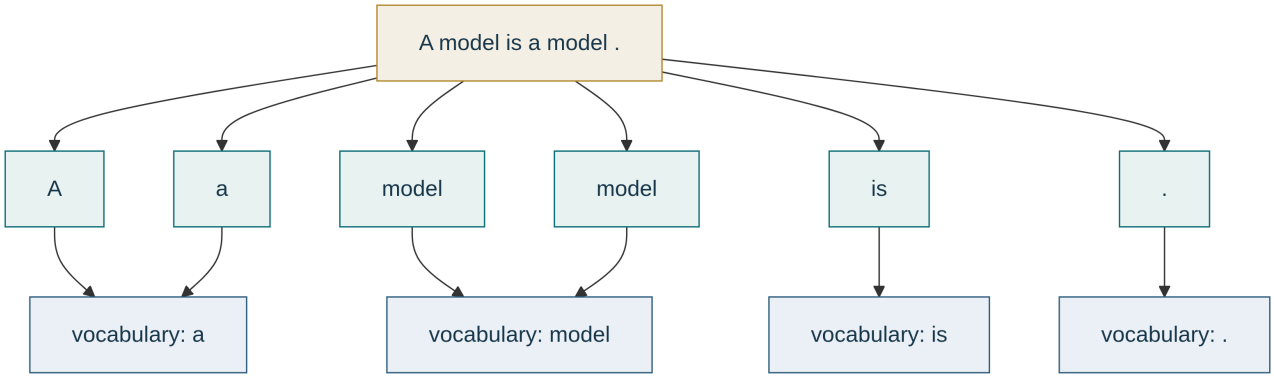


Figure 2: Repeated token instances point to the same vocabulary entry.

2 Why not tokenize into characters?

A character vocabulary is small and can spell any new word.

```
"token" → ["t", "o", "k", "e", "n"]
```

This avoids unknown words and preserves spelling. It also creates long sequences whose individual units carry little meaning. A model must learn that five distant character decisions together form the familiar concept *token*.

For modern sequence models, length matters: longer sequences require more computation and make long-range relationships harder to manage.

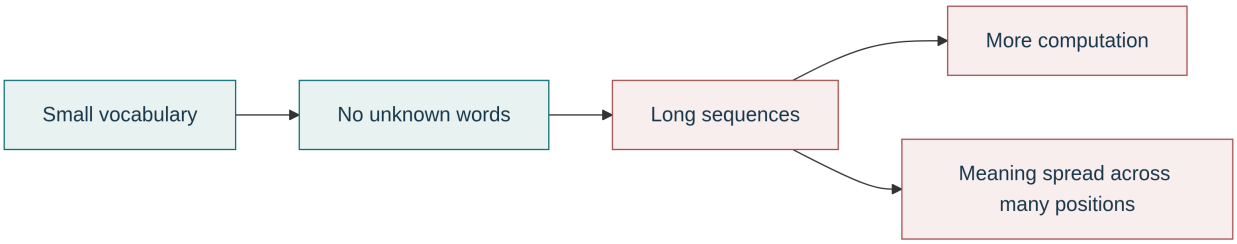


Figure 3: Character tokenization buys universal coverage at the cost of long sequences.

3 Why not tokenize into words?

Word tokens are intuitive and produce shorter sequences:

```
"Tokenization is useful" → ["Tokenization", "is", "useful"]
```

But language continually creates new names, inflections, compounds, technical terms, hashtags, and spelling variants. A fixed vocabulary can never contain every word form.

```
train         known
trained       perhaps known
retrained     perhaps unknown
retraining    perhaps unknown
```

A word-level system either grows an enormous vocabulary or maps unfamiliar words to a single `<UNK>` token. Both choices waste useful structure.

3.1 The granularity dilemma

Unit	Vocabulary	Sequence	New words	Main weakness
Character	Small	Long	Always representable	Meaning is distributed across many positions
Word	Very large	Short	Often unknown	Vocabulary cannot cover productive language

Unit	Vocabulary	Sequence	New words	Main weakness
Subword	Moderate	Moderate	Usually decomposable	Segmentation can be unintuitive

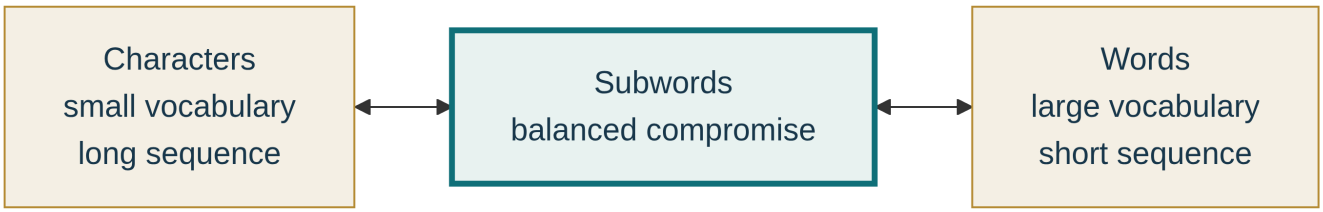


Figure 4: Subwords occupy the useful middle ground between character coverage and word efficiency.

4 The modern compromise: subwords

A **subword tokenizer** keeps frequent forms intact and decomposes rarer forms into reusable pieces.

```
the             → ["the"]
tokenization    → ["token", "ization"]
unhappiness     → ["un", "happi", "ness"]
```

These pieces are learned from frequency patterns, not guaranteed linguistic morphemes. The split `happi + ness` is useful to a model even though `happi` is not an English word.

4.1 The intuition behind byte-pair encoding

Byte-pair encoding (BPE) starts with small units and repeatedly merges frequent neighboring pairs (Sennrich et al. 2016).



Figure 5: A simplified BPE story: frequent adjacent units become reusable vocabulary entries.

We only need the principle today. The complete learning algorithm and its alternatives belong to a deeper treatment of model construction.

4.2 Names you will encounter

- **BPE:** merges frequent adjacent units (Sennrich et al. 2016).
- **WordPiece:** grows a vocabulary using a likelihood-based criterion rather than raw pair frequency (Schuster and Nakajima 2012).
- **Unigram:** begins with many candidate pieces and removes those whose absence least harms the segmentation likelihood (Kudo 2018).
- **SentencePiece:** trains directly from raw text and treats whitespace explicitly (Kudo and Richardson 2018).
- **Byte-level subwords:** begin from a fixed byte vocabulary, guaranteeing coverage, then merge byte sequences for greater efficiency (Wang et al. 2019).

Different algorithms and vocabularies split the same text differently. A tokenizer is therefore part of its model—not a detachable generic preprocessing step.

5 Tokenization in practice

5.1 A deliberately small regular-expression tokenizer

Regular expressions are useful when the task has a small, explicit token definition.

```
import re

TOKEN_PATTERN = r"\b\w+(?:['’]\w+)?\b|[^\w\s]"

text = "NLP isn't just words—it includes punctuation!"
tokens = re.findall(TOKEN_PATTERN, text)
print(tokens)
```

```
['NLP', "isn't", 'just', 'words', '—', 'it', 'includes', 'punctuation',
↪  '!']
```

This is readable and testable, but it is not a universal tokenizer. URLs, abbreviations, emoji sequences, scripts without whitespace, and language-specific conventions quickly require more care.

What the expression says

- `\b\w+` begins with one or more word characters.
- `(?:['’]\w+)?` optionally preserves an apostrophe followed by more word characters.
- `|` means “or.”
- `[^\w\s]` captures a remaining symbol that is neither a word character nor whitespace.

5.2 A linguistic tokenizer with spaCy

spaCy combines general rules with language-specific exceptions for forms such as contractions, abbreviations, punctuation, URLs, and email addresses (Explosion 2026).

```
import spacy

nlp = spacy.blank("en")
examples = [
    "Apple is buying a U.K. startup for $1.2 billion.",
    "Don't email support@example.com—visit https://example.com.",
]

for text in examples:
    print([token.text for token in nlp(text)])
```

The useful classroom question is not merely whether the output “looks right.” Ask which rule produced each boundary and whether those boundaries suit the downstream task.

5.3 A pretrained model tokenizer

Hugging Face’s `AutoTokenizer` loads the exact tokenizer associated with a pretrained model (Hugging Face 2026).

```
from transformers import AutoTokenizer

tokenizer =
↪  AutoTokenizer.from_pretrained("bert-base-multilingual-cased")

examples = [
    "Tokenization is surprisingly complicated.",
    "La tokenisation dépend du modèle.",
    "□□□□□□□ □□□ □□□□□ □□□□□□□.",
]

for text in examples:
    pieces = tokenizer.tokenize(text)
    ids = tokenizer(text, add_special_tokens=False)["input_ids"]
    print(text)
    print(pieces)
    print(ids)
```

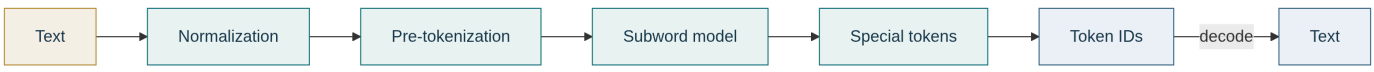


Figure 6: A modern tokenizer performs several coordinated operations and can reverse IDs back into text.

⚠ Never pair a model with a different tokenizer merely because both accept text. Token IDs only have meaning relative to the vocabulary and embedding table used during that model's training.

6 Token IDs are not meanings

Suppose our vocabulary assigns:

```
king  → 341
queen → 520
table → 1208
```

These integers are arbitrary labels. A simple numerical comparison would incorrectly suggest that `queen` lies between `king` and `table`.

One solution is a **one-hot vector** with one coordinate per vocabulary entry:

```
king  → [1, 0, 0, 0]
queen → [0, 1, 0, 0]
table → [0, 0, 1, 0]
```

One-hot vectors distinguish words, but they do not express relationships. The dot product between any two different vectors is zero: *king* is no closer to *queen* than to *table*. This is the central representational limitation emphasized in the opening CS224N notes (Hewitt 2023).

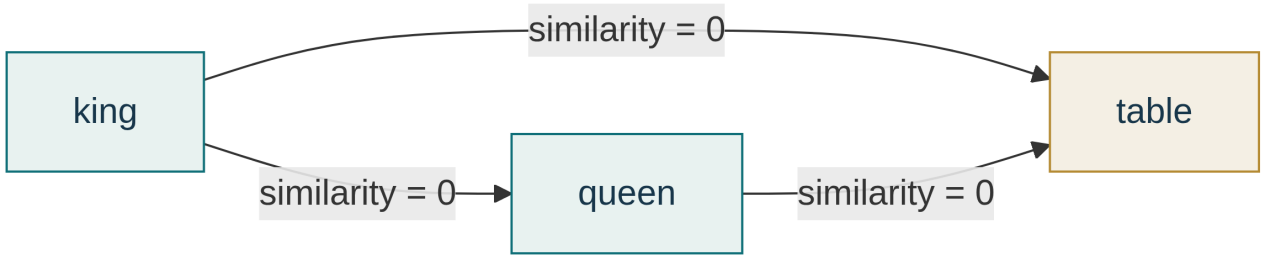


Figure 7: In a one-hot representation, every distinct word occupies an independent axis.

7 Embeddings: meaning as geometry

An **embedding** maps each vocabulary item to a dense, low-dimensional vector.

```
king  → [ 0.21, -0.48,  0.73, ...]
queen → [ 0.24, -0.41,  0.69, ...]
table → [-0.62,  0.18,  0.07, ...]
```

The individual coordinates are usually not named properties. Their value comes from the geometry of the whole space: words used in similar contexts tend to occupy nearby regions.

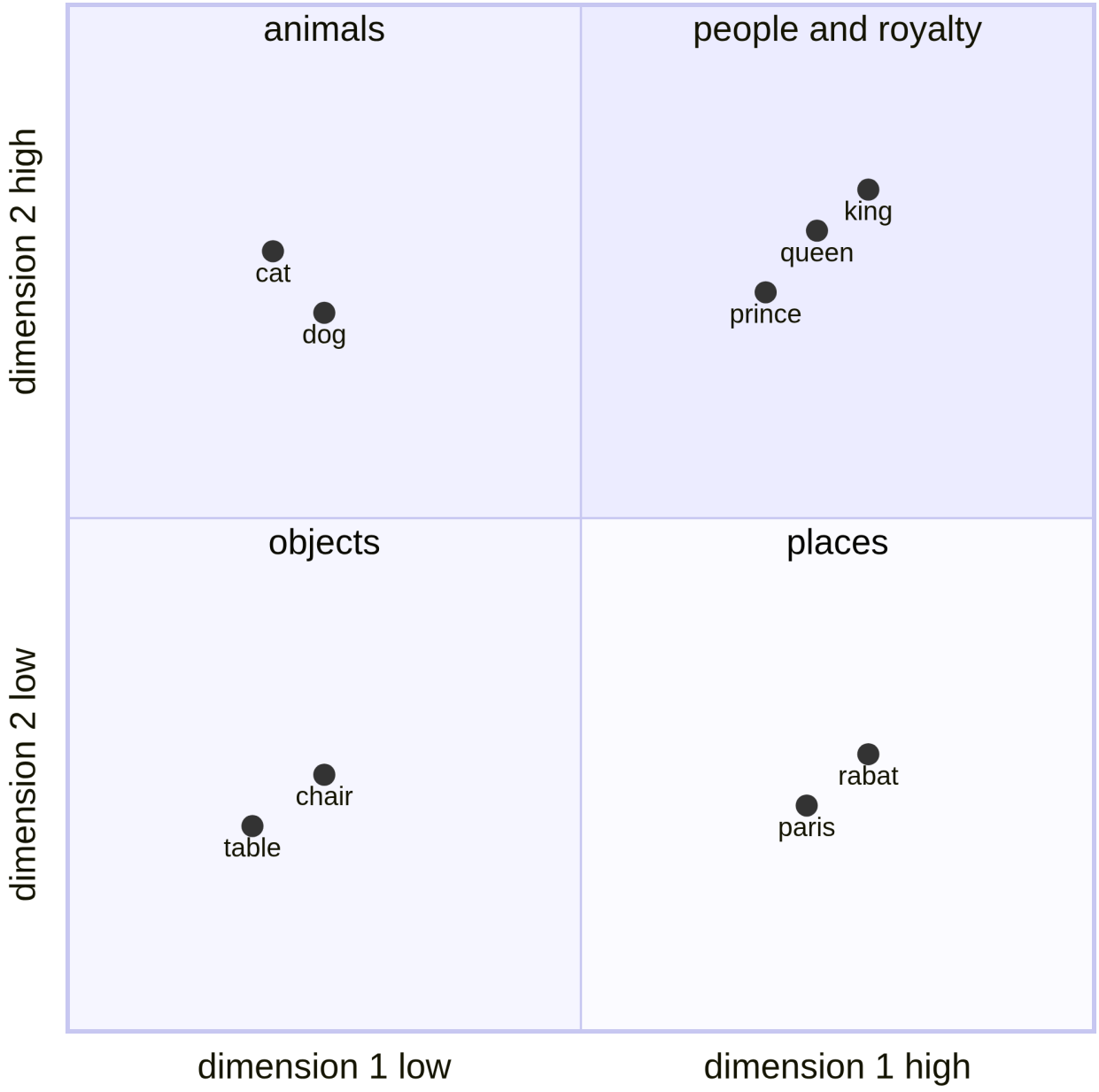


Figure 8: Dense embeddings can place related words near one another while separating unrelated concepts.

> The diagram is schematic. Real embeddings may have hundreds of dimensions; a two-dimensional projection is useful for exploration but cannot preserve every distance.

7.1 Measuring similarity

Cosine similarity compares the direction of two vectors rather than their absolute length:

$$\operatorname{cosine}(\mathbf{u}, \mathbf{v}) = \frac{\mathbf{u}^{\top}\mathbf{v}}{\|\mathbf{u}\|\,\|\mathbf{v}\|}.$$

Values near `1` indicate similar directions; values near `0` indicate little directional relationship.

```
similarity(cat, dog)  >  similarity(cat, democracy)
```

7.2 The classical analogy

A famous demonstration uses vector arithmetic:

$$\textbf{king} - \textbf{man} + \textbf{woman} \approx \textbf{queen}.$$

The relation from *man* to *woman* is approximately repeated from *king* to *queen*. This is an illuminating regularity, not a universal algebra of meaning. Results depend on the corpus, algorithm, vocabulary, and similarity rule—and the same space can encode harmful social stereotypes.

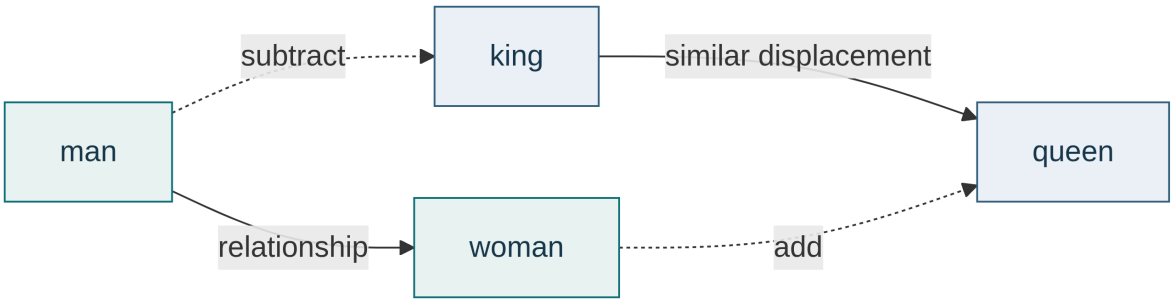


Figure 9: The classical analogy asks for a similar displacement in embedding space.

8 Exploring pretrained embeddings with Gensim

Gensim's `KeyedVectors` provides vector lookup, cosine similarity, nearest neighbors, and analogy queries without requiring the original training model (Řehůřek 2026).

Install the demonstration dependencies in a course project:

```
uv add gensim scikit-learn matplotlib
```

The following loads 100-dimensional GloVe vectors. The first execution downloads the model; later executions use the local cache.

```
import gensim.downloader as api

vectors = api.load("glove-wiki-gigaword-100")

print(vectors.similarity("cat", "dog"))
print(vectors.similarity("cat", "democracy"))
print(vectors.most_similar("language", topn=5))
print(
    vectors.most_similar(
        positive=["king", "woman"],
        negative=["man"],
        topn=5,
    )
)
```

The official Gensim example returns `queen` as the leading answer for the analogy, with cosine similarity around `0.77` for this pretrained model (Řehůřek 2026). Treat the neighbors as evidence to inspect, not proof that the vectors understand the concepts.

8.1 Plot a small part of the embedding space

We cannot directly draw 100 dimensions. Principal component analysis (PCA) projects the selected vectors into two dimensions for inspection.

```
import matplotlib.pyplot as plt
from sklearn.decomposition import PCA

words = [
    "king", "queen", "prince", "princess",
    "man", "woman", "boy", "girl",
    "cat", "dog", "paris", "france",
]

matrix = vectors[words]
points = PCA(n_components=2).fit_transform(matrix)

fig, ax = plt.subplots(figsize=(9, 6))
ax.scatter(points[:, 0], points[:, 1], color="#0f6f78")

for word, (x, y) in zip(words, points):
    ax.annotate(word, (x, y), xytext=(5, 4), textcoords="offset
 ↪  points")

ax.axhline(0, color="#d9e3e1", linewidth=0.8)
ax.axvline(0, color="#d9e3e1", linewidth=0.8)
ax.set_title("A two-dimensional PCA projection of selected GloVe
 ↪  vectors")
ax.set_xlabel("Principal component 1")
ax.set_ylabel("Principal component 2")
```

```
plt.tight_layout()
plt.show()
```

⚠ PCA preserves directions of greatest variance, not every pairwise distance. Moving the selected words can change the projection. Always verify claims with similarities in the original embedding space.

9 What comes next?

We now know what a useful embedding should accomplish, but we have not explained where its coordinates come from.

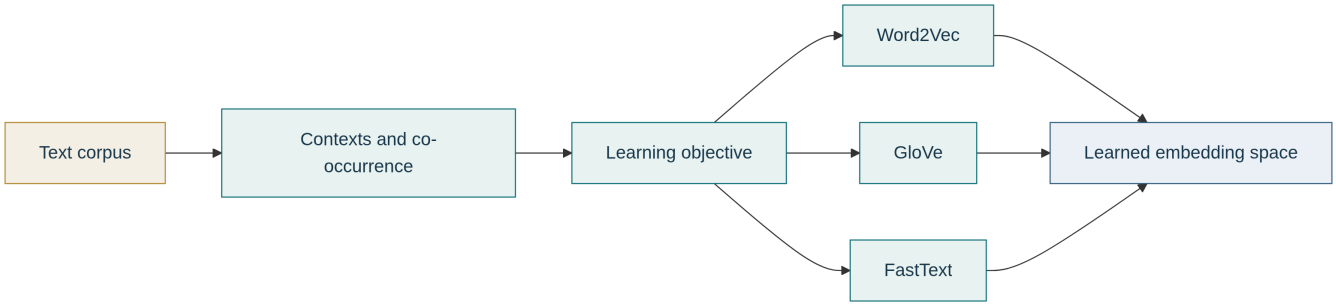


Figure 10: Chapter 3 moves from inspecting embeddings to learning them from patterns of context.

The next chapter develops the distributional hypothesis—*a word is known by the company it keeps*—then derives classical and more recent methods for learning word representations efficiently.

10 Further reading and tools

10.1 Good reads

- Hewitt, *CS224N Lecture 1: Introduction and Word2Vec* (Hewitt 2023) — the representation problem, one-hot vectors, and the motivation for learned embeddings.
- Sennrich, Haddow, and Birch, *Neural Machine Translation of Rare Words with Subword Units* (Sennrich et al. 2016) — the influential adaptation of BPE to open-vocabulary neural NLP.
- Schuster and Nakajima, *Japanese and Korean Voice Search* (Schuster and Nakajima 2012) — the original WordPiece formulation.
- Kudo, *Subword Regularization* (Kudo 2018) — the unigram language-model tokenizer and sampling alternative segmentations.
- Kudo and Richardson, *SentencePiece* (Kudo and Richardson 2018) — language-independent subword training directly from raw text.
- Wang, Cho, and Gu, *Neural Machine Translation with Byte-Level Subwords* (Wang et al. 2019) — byte-level BPE for compact, open-vocabulary multilingual modeling.

- Mielke et al., *Between Words and Characters* (Mielke et al. 2021) — an excellent broader history of open-vocabulary modeling and tokenization.

10.2 Libraries used in this chapter

- Python `re` documentation — bounded rule-based text extraction.
- spaCy tokenization documentation — language rules, exceptions, and token-boundary debugging.
- Hugging Face tokenizer documentation — model tokenizers, IDs, special tokens, encoding, and decoding.
- Gensim `KeyedVectors` documentation — pretrained vectors, similarity, neighbors, and analogies.
- scikit-learn PCA documentation — dimensionality reduction for the visualization.

> **Optional technical reference**
>
> The former detailed chapter—including Unicode, normalization, grapheme clusters, offsets, and reproducible preprocessing design—has been preserved as an optional technical reference.

Explosion. 2026. *Linguistic Features: Tokenization*. https://spacy.io/usage/linguistic-features#tokenization.

Hewitt, John. 2023. *CS224N Lecture 1 Notes: Introduction and Word2Vec*. Stanford University. https://web.stanford.edu/class/cs224n/readings/cs224n_winter2023_lecture1_notes_draft.pdf.

Hugging Face. 2026. *Tokenizers*. https://huggingface.co/docs/transformers/main/fast_tokenizers.

Kudo, Taku. 2018. "Subword Regularization: Improving Neural Network Translation Models with Multiple Subword Candidates." *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics*, 66–75. https://doi.org/10.18653/v1/P18-1007.

Kudo, Taku, and John Richardson. 2018. "SentencePiece: A Simple and Language Independent Subword Tokenizer and Detokenizer for Neural Text Processing." *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, 66–71. https://doi.org/10.18653/v1/D18-2012.

Mielke, Sabrina J., Zaid Alyafeai, Elizabeth Salesky, et al. 2021. "Between Words and Characters: A Brief History of Open-Vocabulary Modeling and Tokenization in NLP." *Transactions of the Association for Computational Linguistics* 9: 1572–85. https://doi.org/10.1162/tacl_a_00429.

Řehůřek, Radim. 2026. *KeyedVectors: Store and Query Word Vectors*. https://radimrehurek.com/gensim/models/keyedvectors.html.

Schuster, Mike, and Kaisuke Nakajima. 2012. "Japanese and Korean Voice Search." *2012 IEEE International Conference on Acoustics, Speech and Signal Processing*, 5149–52.

https://doi.org/10.1109/ICASSP.2012.6289079.

Sennrich, Rico, Barry Haddow, and Alexandra Birch. 2016. “Neural Machine Translation of Rare Words with Subword Units.” *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics*, 1715–25. https://doi.org/10.18653/v1/P16-1162.

Wang, Changhan, Kyunghyun Cho, and Jiatao Gu. 2019. “Neural Machine Translation with Byte-Level Subwords.” *arXiv Preprint arXiv:1909.03341*, ahead of print. https://doi.org/10.48550/arXiv.1909.03341.